Year II — Computational Mathematics

Based on lectures by Various Lecturers

Notes taken by Anonymous

Michaelmas 2026

Contents

1 Numerical and symbolic Computing **2**
1.1 Representations of Numbers . . . 2
1.1.1 Symbolic and Numerical Representations . . . 2
1.1.2 Integer Representations . . . 3
1.1.3 Fixed-Point Numbers . . . 3
1.1.4 Floating-Point Numbers . . . 4
1.1.5 Normalisation . . . 4
1.1.6 IEEE 754 Double Precision . . . 5
1.2 Rounding and Numerical Errors . . . 5
1.2.1 Rounding . . . 5
1.2.2 Significant Figures . . . 6
1.2.3 Absolute and Relative Errors . . . 6
1.2.4 Relative Error in Floating-Point Rounding . . . 6
1.2.5 Unit Roundoff and Machine Epsilon . . . 7
1.3 Overflow and Underflow . . . 7
1.4 Loss of Significance . . . 8
1.5 Revision Checklist . . . 9
1.5.1 Key Topics . . . 9
1.5.2 Key Skills . . . 9

2 Continuous Functions **10**
2.1 Interpolation . . . 10
2.1.1 Motivation and Taylor Approximation . . . 10
2.1.2 Polynomial Interpolation . . . 10
2.1.3 Lagrange Interpolation . . . 11
2.1.4 Interpolation Error . . . 12
2.1.5 Runge Phenomenon and Chebyshev Nodes . . . 12
2.2 Non-Linear Equations . . . 13
2.2.1 Problem and Motivation . . . 13
2.2.2 Bisection Method . . . 13
2.2.3 Fixed-Point Iteration . . . 14
2.2.4 Order of Convergence . . . 15
2.2.5 Newton's Method . . . 15
2.2.6 Newton's Method for Systems . . . 16
2.2.7 Secant Method . . . 17
2.2.8 Comparison of Root-Finding Methods . . . 17
2.2.9 Problem-Solving Checklist . . . 17

Chapter 1

Numerical and symbolic Computing

“‘latex

1.1 Representations of Numbers

1.1.1 Symbolic and Numerical Representations

Computers represent and manipulate mathematical objects using finite amounts of memory. There are two broad approaches to computation:

Definition 1.1.1 (Symbolic Computation)**.** Symbolic computation manipulates mathematical expressions exactly, retaining their algebraic structure. For example,

$$\frac{1}{3}+\frac{1}{6}=\frac{1}{2}.$$

Computer Algebra Systems (CAS), such as MATLAB's Symbolic Math Toolbox, can perform symbolic differentiation, integration and algebraic simplification.

Definition 1.1.2 (Numerical Computation)**.** Numerical computation represents quantities using finite numerical formats and approximates mathematical operations. For example, a computer may store an approximation to $\frac{1}{3}$ rather than its exact value.

Remark 1.1.3 (Symbolic vs Numerical Computation)**.**

- Symbolic computation can preserve exact results, but expressions may become expensive to manipulate as their complexity increases.
- Numerical computation is generally more efficient for large-scale calculations, but finite precision introduces rounding errors.
- Symbolic and numerical methods are complementary: symbolic computation can derive or simplify formulas, while numerical computation can evaluate them efficiently.

Example 1.1.4. In MATLAB, compare

```
0.1 + 0.2 == 0.3
```

with

```
sym(1)/10 + sym(2)/10 == sym(3)/10
```

The first comparison can return false because 0.1 and 0.2 do not have exact finite binary floating-point representations. The symbolic calculation retains the exact rational values and returns true.

1.1.2 Integer Representations

Integers are represented using a finite number of binary digits, called bits. Each bit stores either 0 or 1.

An unsigned integer represented using w bits has the range

$$\boxed{0 \leq n \leq 2^w - 1.}$$

For example, using four bits,

$$(1011)_2 = 1 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 11.$$

Signed integers require a convention for representing negative values. A common representation is *two's complement*. Using w bits, its range is

$$\boxed{-2^{w-1} \leq n \leq 2^{w-1} - 1.}$$

Remark 1.1.5 (Integer Overflow)**.** Integer arithmetic can produce a mathematical result outside the range representable by the chosen integer type. This is called integer overflow. The resulting behaviour depends on the programming language and numeric type; fixed-width integer arithmetic may wrap around or raise an error.

1.1.3 Fixed-Point Numbers

Definition 1.1.6 (Fixed-Point Numbers)**.** A fixed-point number represents a number using a fixed number of digits before and after the radix point. In base β, a finite representation can be written as

$$x = \pm(d_1 d_2 \cdots d_{k-1}.d_k \cdots d_n)_\beta,$$

where

$$d_i \in \{0, 1, \ldots, \beta - 1\}.$$

The base β determines the number system: $\beta = 10$ gives decimal and $\beta = 2$ gives binary.

For a fixed-point system with q digits after the radix point, the spacing between adjacent representable numbers is

$$\boxed{\Delta = \beta^{-q}.}$$

For example, a decimal fixed-point system with two digits after the decimal point has spacing

$$\Delta = 10^{-2} = 0.01.$$

Remark 1.1.7 (Properties of Fixed-Point Numbers)**.**

- The position of the radix point is fixed.
- Representable values are equally spaced.
- The spacing determines the resolution of the representation.
- The number of digits before the point limits the magnitude of representable values.
- Values that do not lie on the fixed-point grid must be rounded or truncated.

Example 1.1.8. Suppose a decimal fixed-point system stores two digits after the decimal point. Then

$$3.14$$

is representable exactly, whereas

$$3.14159$$

must be approximated, for example by 3.14 when rounded to two decimal places.

1.1.4 Floating-Point Numbers

Fixed-point numbers have constant spacing, which limits the range of magnitudes that can be represented conveniently. Floating-point numbers address this by allowing the radix point to move through the use of an exponent.

Definition 1.1.9 (Floating-Point Numbers)**.** A floating-point system is specified by:

- a base $\beta \geq 2$;
- a significand precision of m digits;
- a permitted range of exponents.

A finite floating-point number can be written as

$$\boxed{x = \pm(0.d_1 d_2 \cdots d_m)_\beta \, \beta^e,}$$

where $d_i \in \{0, 1, \ldots, \beta - 1\}$ and e is an integer in the permitted exponent range.

The digits $d_1, \ldots, d_m$ form the *significand*, also called the fraction or mantissa, and e is the exponent.

Example 1.1.10. Consider a decimal floating-point system with three significand digits and exponents between -2 and 2. It can represent numbers such as

$$0.123 \times 10^2 = 12.3,$$

$$0.456 \times 10^{-2} = 0.00456,$$

and

$$-0.789 \times 10^1 = -7.89.$$

However, 12.3456 cannot be represented exactly with only three significand digits.

Example 1.1.11. In binary,

$$(0.101)_2 = 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = \frac{5}{8}.$$

Hence

$$(0.101)_2 \, 2^2 = \frac{5}{8} \cdot 4 = 2.5.$$

Similarly,

$$(0.100)_2 \, 2^1 = \frac{1}{2} \cdot 2 = 1.$$

1.1.5 Normalisation

Definition 1.1.12 (Normalised Floating-Point Numbers)**.** A non-zero floating-point number is normalised when its significand is written so that its first digit is non-zero:

$$x = \pm(0.d_1 d_2 \cdots d_m)_\beta \, \beta^e, \qquad d_1 \neq 0.$$

Normalisation gives a standard representation and maximises the precision available for a given number of significand digits. Unlike fixed-point representation, floating-point spacing changes with the magnitude of the number.

Remark 1.1.13 (Spacing Between Floating-Point Numbers)**.** For a normalised base-β system with m significand digits, the spacing between adjacent numbers is proportional to the magnitude of the numbers being represented. Numbers of larger magnitude generally have larger absolute spacing, while the relative spacing remains approximately constant within a fixed exponent range.

1.1.6 IEEE 754 Double Precision

MATLAB normally uses IEEE 754 double-precision floating-point arithmetic for standard numerical calculations.

A double-precision number occupies 64 bits:

- 1 sign bit;
- 11 exponent bits;
- 52 explicitly stored fraction bits.

For normalised numbers, the leading binary digit is implicitly 1. If E is the stored exponent, then

$$\boxed{x = (-1)^s (1.d_1 d_2 \cdots d_{52})_2 \, 2^{E-1023},}$$

where s is the sign bit and $1 \leq E \leq 2046$.

Equivalently,

$$x = (-1)^s (0.1 d_1 d_2 \cdots d_{52})_2 \, 2^{E-1022}.$$

Remark 1.1.14 (Exponent Bias)**.** The stored exponent is biased by 1023. For normalised double-precision numbers, the unbiased exponent therefore ranges from -1022 to 1023 in the conventional $1.d_1 \cdots d_{52}$ representation.

The approximate limits of double precision are

$$\boxed{\texttt{realmin} \approx 2.2251 \times 10^{-308}}$$

and

$$\boxed{\texttt{realmax} \approx 1.7977 \times 10^{308}.}$$

In MATLAB, these values can be inspected using

realmin
realmax

Remark 1.1.15 (Subnormal Numbers and Special Values)**.** IEEE 754 reserves the extreme exponent fields for special cases:

- A zero exponent field represents signed zero or a subnormal number.
- An all-ones exponent field represents infinity or NaN (*not a number*).

Subnormal numbers extend the range of representable non-zero values below the smallest positive normal number, but with reduced precision.

1.2 Rounding and Numerical Errors

1.2.1 Rounding

A computer cannot store every real number exactly using a finite floating-point format. Rounding maps a real number to a representable floating-point number.

Definition 1.2.1 (Floating-Point Rounding)**.** The rounded floating-point representation of a real number x is denoted by

$$\boxed{\mathrm{fl}(x).}$$

Under round-to-nearest arithmetic, $\mathrm{fl}(x)$ is a representable number closest to x. When x lies exactly

halfway between two representable values, IEEE 754 normally rounds to the value whose last significand digit is even.

Remark 1.2.2 (Why Round to Nearest Even?)**.** Ties-to-even rounding helps avoid a systematic bias that could arise from always rounding halfway cases in the same direction.

1.2.2 Significant Figures

Definition 1.2.3 (Significant Figures)**.** Significant figures begin at the first non-zero digit and include all subsequent digits. Trailing zeros after a decimal point count as significant when they indicate stated precision.

Example 1.2.4. The number 0.004560 has four significant figures:

$$4, \quad 5, \quad 6, \quad 0.$$

The leading zeros locate the decimal point and are not significant.

1.2.3 Absolute and Relative Errors

Definition 1.2.5 (Absolute Error)**.** If x is the true value and $\hat{x}$ is an approximation, the absolute error is

$$\boxed{E_{\text{abs}} = |\hat{x} - x|.}$$

Definition 1.2.6 (Relative Error)**.** Provided $x \neq 0$, the relative error is

$$\boxed{E_{\text{rel}} = \frac{|\hat{x} - x|}{|x|}.}$$

The percentage relative error is $100E_{\text{rel}}\%$.

Remark 1.2.7 (Interpreting Error)**.** Absolute error measures the size of the discrepancy in the original units. Relative error measures the discrepancy compared with the magnitude of the true value, making it useful for comparing accuracy across different scales.

Example 1.2.8. Suppose the true value is $x = 1000$ and the approximation is $\hat{x} = 999$. Then

$$E_{\text{abs}} = 1, \qquad E_{\text{rel}} = \frac{1}{1000} = 0.001.$$

The percentage relative error is 0.1%.

1.2.4 Relative Error in Floating-Point Rounding

For a non-zero value whose rounded result is finite and non-zero, we can write

$$\boxed{\text{fl}(x) = x(1 + \delta),}$$

where

$$\boxed{\delta = \frac{\text{fl}(x) - x}{x}.}$$

The quantity δ is the signed relative rounding error. Its magnitude is the relative error introduced by rounding.

1.2.5 Unit Roundoff and Machine Epsilon

Definition 1.2.9 (Unit Roundoff)**.** Consider a normalised floating-point system with base β and m significand digits. Under round-to-nearest arithmetic, provided the result remains in the normal range,

$$\mathrm{fl}(x) = x(1+\delta),$$

where

$$\boxed{|\delta| \le u, \qquad u = \frac{1}{2}\beta^{1-m}.}$$

The constant u is called the *unit roundoff*.

The bound follows because rounding changes the significand by at most half a unit in its final digit. The first digit of a normalised significand is non-zero, so its magnitude is at least β^{-1}. Comparing the absolute rounding error with the magnitude of x gives

$$|\delta| \le \frac{1}{2}\beta^{1-m}.$$

Remark 1.2.10 (Unit Roundoff vs Machine Epsilon)**.** These quantities are related but are not always defined identically:

- **Unit roundoff** u is the standard upper bound on relative error under round-to-nearest arithmetic.
- **Machine epsilon** is commonly used to describe the spacing between 1 and the next larger representable number.

For IEEE 754 double precision,

$$\boxed{u = 2^{-53} \approx 1.11 \times 10^{-16},}$$

whereas MATLAB's

$$\boxed{\texttt{eps} = 2^{-52} \approx 2.22 \times 10^{-16}.}$$

Thus, for MATLAB double precision, `eps` is twice the unit roundoff.

Remark 1.2.11 (Correctly Rounded Arithmetic)**.** For the basic operations $+, -, \times, \div$, IEEE 754 correctly rounded arithmetic produces the exact mathematical result rounded to the floating-point format, subject to the relevant exceptional cases. When the exact result is in the normal range and no exceptional behaviour occurs,

$$\boxed{\mathrm{fl}(x \circ y) = (x \circ y)(1+\delta), \qquad |\delta| \le u,}$$

where $\circ$ denotes one of these operations and the exact result is non-zero.

1.3 Overflow and Underflow

Definition 1.3.1 (Overflow)**.** Overflow occurs when a computed result exceeds the largest finite magnitude representable in the floating-point format. In standard IEEE 754 arithmetic, the result of an overflowing operation is typically $+\infty$ or $-\infty$, depending on the sign.

Example 1.3.2. In MATLAB, a sufficiently large calculation can produce infinity:

```
2 * realmax
```

Definition 1.3.3 (Underflow)**.** Underflow occurs when a non-zero result is too small in magnitude to be represented with normal floating-point precision. IEEE 754 supports subnormal numbers to represent some values below the smallest positive normal number. Values smaller still may round to zero.

Remark 1.3.4 (Managing Overflow and Underflow)**.**

- Avoid forming unnecessarily large intermediate quantities.
- Rescale calculations where appropriate.
- Use numerically stable formulas when available.
- Check for non-finite results using MATLAB's `isinf` and `isfinite`.
- Remember that subnormal numbers have reduced precision; a non-zero result is not necessarily a normally represented result.

1.4 Loss of Significance

Definition 1.4.1 (Loss of Significance)**.** Loss of significance occurs when numerical errors become large relative to the final answer. A common cause is subtracting two nearly equal numbers, which is known as *catastrophic cancellation*.

Suppose the stored inputs are

$$\hat{x} = x(1+\delta_1), \qquad \hat{y} = y(1+\delta_2).$$

Then

$$\hat{x} \pm \hat{y} = x \pm y + x\delta_1 \pm y\delta_2.$$

Therefore, the error inherited from the input approximations is

$$x\delta_1 \pm y\delta_2.$$

Relative to the exact result, the corresponding relative error is

$$\boxed{\frac{x\delta_1 \pm y\delta_2}{x \pm y}},$$

provided $x \pm y \neq 0$.

If x and y are nearly equal and are subtracted, the denominator can be very small compared with the individual terms. Consequently, even small input errors can produce a large relative error in the result.

Example 1.4.2 (Catastrophic Cancellation)**.** Consider the equation

$$x^2 - 56x + 1 = 0.$$

By the quadratic formula,

$$x_1 = 28 + \sqrt{783}, \qquad x_2 = 28 - \sqrt{783}.$$

To four significant figures,

$$\sqrt{783} \approx 27.98.$$

Direct calculation then gives

$$\hat{x}_1 = 28 + 27.98 = 55.98,$$

but

$$\hat{x}_2 = 28 - 27.98 = 0.02000.$$

The smaller root is inaccurate because two nearly equal numbers have been subtracted.

A more stable approach uses the fact that the product of the roots is 1:

$$x_1 x_2 = 1.$$

Hence

$$x_2 = \frac{1}{x_1} \approx \frac{1}{55.98} \approx 0.01786.$$

This avoids the cancellation in the direct formula.

Remark 1.4.3 (Reducing Loss of Significance)**.**

- Avoid subtracting nearly equal quantities when an algebraically equivalent alternative is available.
- Reformulate calculations to avoid cancellation.
- Use stable algorithms rather than relying solely on algebraic equivalence.
- Increasing precision can help, but does not necessarily fix an unstable method.

1.5 Revision Checklist

Use this checklist to assess whether you understand the chapter and can apply its main ideas.

1.5.1 Key Topics

- ☐ Explain integer, fixed-point and floating-point representations.
- ☐ Describe how binary digits encode integers and how the number of bits limits the representable range.
- ☐ Explain the role of the significand, exponent, sign bit and normalisation in floating-point representation.
- ☐ Distinguish absolute error from relative error and calculate both.
- ☐ Explain rounding, unit roundoff and machine epsilon, including the difference between MATLAB's `eps` and unit roundoff.
- ☐ Explain overflow, underflow, subnormal numbers and loss of significance.
- ☐ Distinguish symbolic computation from numerical computation and explain their respective advantages.

1.5.2 Key Skills

- ☐ Convert between binary and decimal representations and determine whether a number can be represented exactly in a given format.
- ☐ Determine the range and spacing of values in a specified fixed-point or floating-point system.
- ☐ Estimate absolute and relative rounding errors.
- ☐ Apply the unit-roundoff bound to reason about the accuracy of arithmetic operations.
- ☐ Identify potential overflow and underflow in a numerical calculation.
- ☐ Diagnose catastrophic cancellation and reformulate a calculation to improve numerical stability.
- ☐ Decide whether a symbolic or numerical approach is more suitable for a given problem.

Chapter 2

Continuous Functions

2.1 Interpolation

2.1.1 Motivation and Taylor Approximation

Interpolation is the process of constructing a function, usually a polynomial, that passes through a given set of data points. It is useful when a function is known only at finitely many points and we want to estimate its values elsewhere.

Remark 2.1.1 (Taylor Approximation vs Interpolation)**.** Taylor polynomials match the derivatives of a function at a single point. Interpolating polynomials match the values of a function at several distinct points. These are different constructions, and their error formulas are different.

Theorem 2.1.2 (Weierstrass Approximation Theorem)**.** If f is continuous on a closed interval $[a, b]$, then for every $\varepsilon > 0$, there exists a polynomial p such that

$$|f(x) - p(x)| < \varepsilon \qquad \text{for all } x \in [a, b].$$

Remark 2.1.3. The Weierstrass approximation theorem guarantees that a sufficiently accurate polynomial approximation exists. It does not guarantee that interpolating polynomials through arbitrary nodes converge as their degree increases.

2.1.2 Polynomial Interpolation

Suppose we are given $n + 1$ distinct nodes

$$x_0, x_1, \ldots, x_n$$

and corresponding function values

$$f(x_0), f(x_1), \ldots, f(x_n).$$

We seek a polynomial p_n of degree at most n satisfying

$$p_n(x_i) = f(x_i), \qquad i = 0, 1, \ldots, n.$$

Theorem 2.1.4 (Existence and Uniqueness of the Interpolating Polynomial)**.** Given $n + 1$ distinct nodes and prescribed function values, there exists a unique polynomial of degree at most n that interpolates the data.

Proof. Write

$$p_n(x) = a_0 + a_1 x + \cdots + a_n x^n.$$

The interpolation conditions give the linear system

$$\begin{pmatrix} 1 & x_0 & \cdots & x_0^n \\ 1 & x_1 & \cdots & x_1^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \cdots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} f(x_0) \\ f(x_1) \\ \vdots \\ f(x_n) \end{pmatrix}.$$

The determinant of the Vandermonde matrix is

$$\prod_{0 \le i < j \le n} (x_j - x_i),$$

which is non-zero because the nodes are distinct. Hence the system has a unique solution. □

2.1.3 Lagrange Interpolation

The Lagrange form constructs the interpolating polynomial directly from the interpolation nodes.

Definition 2.1.5 (Lagrange Basis Polynomials)**.** For distinct nodes $x_0, \ldots, x_n$, define

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}, \qquad i = 0, \ldots, n.$$

These satisfy

$$L_i(x_j) = \begin{cases} 1, & i = j, \\ 0, & i \neq j. \end{cases}$$

Theorem 2.1.6 (Lagrange Interpolation Formula)**.** The unique polynomial of degree at most n interpolating f at $x_0, \ldots, x_n$ is

$$\boxed{p_n(x) = \sum_{i=0}^{n} f(x_i) L_i(x).}$$

Example 2.1.7. Find the quadratic interpolating polynomial for

$$f(x) = \cos x$$

at the nodes

$$x_0 = -\frac{\pi}{4}, \qquad x_1 = 0, \qquad x_2 = \frac{\pi}{4}.$$

The corresponding function values are

$$f(x_0) = \frac{1}{\sqrt{2}}, \qquad f(x_1) = 1, \qquad f(x_2) = \frac{1}{\sqrt{2}}.$$

By symmetry, the interpolating polynomial is even, so write

$$p_2(x) = 1 + cx^2.$$

Using $p_2(\pi/4) = 1/\sqrt{2}$,

$$1 + c\frac{\pi^2}{16} = \frac{1}{\sqrt{2}}.$$

Therefore,

$$\boxed{p_2(x) = 1 + \frac{16}{\pi^2}\left(\frac{1}{\sqrt{2}} - 1\right) x^2.}$$

Remark 2.1.8 (Practical Use)**.** For hand calculations, the Lagrange formula is often convenient because it gives the polynomial explicitly. For repeated interpolation with changing data, other forms such as Newton interpolation may be more efficient.

2.1.4 Interpolation Error

The interpolation polynomial generally differs from the original function between the interpolation nodes.

Theorem 2.1.9 (Lagrange Interpolation Error)**.** Suppose $f \in C^{n+1}([a,b])$, and let p_n interpolate f at the distinct nodes $x_0, \dots, x_n \in [a,b]$. For each $x \in [a,b]$, there exists a point $\xi_x \in [a,b]$ such that

$$\boxed{f(x) - p_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).}$$

If

$$|f^{(n+1)}(x)| \le M \qquad \text{for all } x \in [a,b],$$

then

$$\boxed{|f(x) - p_n(x)| \le \frac{M}{(n+1)!} \left| \prod_{i=0}^{n} (x - x_i) \right|.}$$

Example 2.1.10. Suppose $f(x) = \cos x$ is interpolated at

$$x_0 = -\frac{\pi}{4}, \qquad x_1 = 0, \qquad x_2 = \frac{\pi}{4}.$$

Since

$$f^{(3)}(x) = \sin x,$$

we have $|f^{(3)}(x)| \le 1$. Therefore,

$$|f(x) - p_2(x)| \le \frac{1}{6} \left| \left(x + \frac{\pi}{4}\right) x \left(x - \frac{\pi}{4}\right) \right|.$$

On $[-\pi/4, \pi/4]$,

$$\left| \left(x + \frac{\pi}{4}\right) x \left(x - \frac{\pi}{4}\right) \right| = \left| x \left(x^2 - \frac{\pi^2}{16}\right) \right|.$$

A simple, though not necessarily sharp, bound is

$$|x| \le \frac{\pi}{4}, \qquad \left| x^2 - \frac{\pi^2}{16} \right| \le \frac{\pi^2}{16}.$$

Hence

$$\boxed{|f(x) - p_2(x)| \le \frac{\pi^3}{384}.}$$

Remark 2.1.11 (How to Use the Error Formula)**.**

(i) Identify the interpolation degree n.

(ii) Differentiate the function $n+1$ times.

(iii) Find an upper bound M for $|f^{(n+1)}|$ on the interval.

(iv) Bound the product $|\prod_{i=0}^{n}(x - x_i)|$.

(v) Substitute both bounds into the error estimate.

2.1.5 Runge Phenomenon and Chebyshev Nodes

Increasing the degree of an interpolating polynomial does not necessarily improve the approximation everywhere.

Remark 2.1.12 (Runge Phenomenon)**.** Interpolation at equally spaced nodes can produce large oscillations near the endpoints as the polynomial degree increases. This behaviour is known as the Runge phenomenon.

A common remedy is to choose Chebyshev nodes, which cluster near the endpoints of the interpolation interval.

Definition 2.1.13 (Chebyshev Nodes)**.** The $n+1$ Chebyshev nodes on $[-1,1]$ are

$$\boxed{x_k = \cos\left(\frac{(2k+1)\pi}{2(n+1)}\right), \qquad k = 0, \dots, n.}$$

For these nodes, the nodal polynomial satisfies

$$\boxed{\max_{x\in[-1,1]} \left|\prod_{k=0}^{n}(x - x_k)\right| = 2^{-n}.}$$

Consequently, if

$$|f^{(n+1)}(x)| \le M \qquad \text{on } [-1,1],$$

then

$$\boxed{\max_{x\in[-1,1]} |f(x) - p_n(x)| \le \frac{M}{(n+1)!2^n}.}$$

Remark 2.1.14. The error bound depends on both the node locations and the size of the $(n+1)$-st derivative. Chebyshev nodes control the nodal-product factor, but they do not guarantee small error for every function and every degree.

2.2 Non-Linear Equations

2.2.1 Problem and Motivation

We want to find a solution to

$$f(x) = 0.$$

Such a solution is called a root or zero of f.

For many nonlinear functions, an exact solution is unavailable or impractical to calculate. Numerical methods therefore generate a sequence of approximations

$$x_0, x_1, x_2, \dots$$

intended to converge to a root r.

Different methods trade off reliability, convergence speed, computational cost and the amount of information required about f.

2.2.2 Bisection Method

The bisection method repeatedly halves an interval known to contain a root.

Theorem 2.2.1 (Bisection Method)**.** Suppose f is continuous on $[a_0, b_0]$ and

$$f(a_0)f(b_0) < 0.$$

Then f has at least one root in (a_0, b_0). Repeated bisection produces nested intervals containing a root, with interval lengths tending to zero.

At each iteration, calculate

$$m_k = \frac{a_k + b_k}{2}.$$

If $f(m_k) = 0$, the root has been found exactly. Otherwise, retain the half-interval on which the function changes sign:

$$[a_{k+1}, b_{k+1}] = \begin{cases} [a_k, m_k], & f(a_k)f(m_k) < 0, \\ [m_k, b_k], & f(m_k)f(b_k) < 0. \end{cases}$$

Theorem 2.2.2 (Bisection Error Bound)**.** After n bisection steps, the interval length is

$$b_n - a_n = \frac{b_0 - a_0}{2^n}.$$

If $m_n = (a_n + b_n)/2$ is the midpoint of the final interval, then the root r satisfies

$$\boxed{|r - m_n| \leq \frac{b_0 - a_0}{2^{n+1}}.}$$

To guarantee midpoint error at most ε, it is sufficient to choose n such that

$$\frac{b_0 - a_0}{2^{n+1}} \leq \varepsilon.$$

Equivalently,

$$\boxed{n \geq \log_2\left(\frac{b_0 - a_0}{2\varepsilon}\right).}$$

Round up to the next integer if necessary.

Remark 2.2.3 (Advantages and Disadvantages)**.**

- Guaranteed convergence when continuity and a sign-changing initial interval are satisfied.
- The error bound is explicit.
- Convergence is linear, with the interval length halved at every step.
- It does not use derivative information.
- It can be slower than Newton's method or the secant method.

2.2.3 Fixed-Point Iteration

A root-finding problem can sometimes be rewritten in the form

$$x = g(x).$$

A solution of this equation is called a fixed point of g.

The fixed-point iteration is

$$\boxed{x_{n+1} = g(x_n).}$$

Remark 2.2.4. The choice of rearrangement matters. Different functions g can have the same fixed point but very different convergence behaviour.

Theorem 2.2.5 (Contraction Mapping Theorem)**.** Suppose $g : [a, b] \to [a, b]$ and there exists a constant $L < 1$ such that

$$|g(x) - g(y)| \leq L|x - y| \qquad \text{for all } x, y \in [a, b].$$

Then g has a unique fixed point $r \in [a, b]$, and the iteration

$$x_{n+1} = g(x_n)$$

converges to r for every initial value $x_0 \in [a, b]$.

A useful sufficient condition for contraction is

$$|g'(x)| \leq L < 1 \qquad \text{for all } x \in [a, b],$$

provided g maps $[a, b]$ into itself.

Theorem 2.2.6 (A Priori Error Bound)**.** Under the contraction assumptions,

$$\boxed{|x_n - r| \leq \frac{L^n}{1-L}|x_1 - x_0|.}$$

Remark 2.2.7 (Local Convergence)**.** If $g(r) = r$ and $|g'(r)| < 1$, then the fixed point is locally attractive: iterations starting sufficiently close to r converge to it, provided the iteration remains in a region where the contraction condition holds. This local condition is weaker than requiring a contraction on the entire interval.

2.2.4 Order of Convergence

Let $x_n \to r$, where r is the exact solution, and define the error

$$e_n = x_n - r.$$

Definition 2.2.8 (Order of Convergence)**.** The sequence (x_n) converges to r with order $p \geq 1$ if there exists a constant $C > 0$ such that

$$\boxed{\lim_{n\to\infty} \frac{|e_{n+1}|}{|e_n|^p} = C.}$$

Equivalently, asymptotically,

$$|e_{n+1}| \approx C|e_n|^p.$$

- $p = 1$: linear convergence.
- $p > 1$: superlinear convergence.
- $p = 2$: quadratic convergence.

Remark 2.2.9. Linear convergence reduces the error by approximately a constant factor per iteration. Quadratic convergence can produce a dramatic reduction in error once the iterates are sufficiently close to the root.

2.2.5 Newton's Method

Newton's method uses the tangent line to approximate a nonlinear function near the current iterate.

The tangent line to f at x_n is

$$y = f(x_n) + f'(x_n)(x - x_n).$$

Setting $y = 0$ and solving for x gives Newton's iteration.

Definition 2.2.10 (Newton's Method)**.** Given an initial guess x_0, define

$$\boxed{x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.}$$

The method requires $f'(x_n) \neq 0$ at each iteration.

Theorem 2.2.11 (Local Quadratic Convergence of Newton's Method)**.** Suppose $f(r) = 0$, $f'(r) \neq 0$, and f is sufficiently smooth near r. If x_0 is sufficiently close to r, then Newton's method converges locally to r, with quadratic convergence under the standard regularity assumptions.

For a simple root, the error satisfies asymptotically

$$\boxed{e_{n+1} \approx \frac{f''(r)}{2f'(r)} e_n^2.}$$

Example 2.2.12. Apply Newton's method to solve

$$f(x) = x^2 - 2 = 0.$$

Since $f'(x) = 2x$, the iteration is

$$x_{n+1} = x_n - \frac{x_n^2 - 2}{2x_n} = \frac{1}{2}\left(x_n + \frac{2}{x_n}\right).$$

Starting with $x_0 = 1$,

$$x_1 = 1.5, \qquad x_2 \approx 1.416667, \qquad x_3 \approx 1.414216.$$

The iterates converge to $\sqrt{2}$.

Remark 2.2.13 (Advantages and Disadvantages)**.**

- Usually very fast near a simple root.
- Has quadratic local convergence under suitable conditions.
- Requires evaluation of the derivative.
- Can fail if $f'(x_n) = 0$ or becomes very small.
- Convergence is not guaranteed for an arbitrary initial guess.

2.2.6 Newton's Method for Systems

For a system of m nonlinear equations in m unknowns, write

$$\mathbf{F}(\mathbf{x}) = \mathbf{0},$$

where

$$\mathbf{x} = \begin{pmatrix} x_1 \\ \vdots \\ x_m \end{pmatrix}, \qquad \mathbf{F}(\mathbf{x}) = \begin{pmatrix} F_1(\mathbf{x}) \\ \vdots \\ F_m(\mathbf{x}) \end{pmatrix}.$$

The Jacobian matrix is

$$J_{\mathbf{F}}(\mathbf{x}) = \begin{pmatrix} \dfrac{\partial F_1}{\partial x_1} & \cdots & \dfrac{\partial F_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \dfrac{\partial F_m}{\partial x_1} & \cdots & \dfrac{\partial F_m}{\partial x_m} \end{pmatrix}.$$

Definition 2.2.14 (Newton's Method for Systems)**.** At each iteration, solve

$$J_{\mathbf{F}}(\mathbf{x}_n)\Delta\mathbf{x}_n = -\mathbf{F}(\mathbf{x}_n)$$

for $\Delta\mathbf{x}_n$, then update

$$\boxed{\mathbf{x}_{n+1} = \mathbf{x}_n + \Delta\mathbf{x}_n.}$$

Remark 2.2.15. In numerical computation, solve the linear system for $\Delta\mathbf{x}_n$ rather than explicitly calculating $J_{\mathbf{F}}(\mathbf{x}_n)^{-1}$. The Jacobian must be nonsingular at the solution for the standard local convergence result to apply.

Example 2.2.16. In MATLAB, a Newton step for a system can be implemented as follows:

```
delta = -J(x) \ F(x);
x_new = x + delta;
```

Here, `F(x)` returns the vector of function values and `J(x)` returns the Jacobian matrix. The backslash operator solves the linear system.

2.2.7 Secant Method

The secant method approximates the derivative in Newton's method using a finite difference:

$$f'(x_n) \approx \frac{f(x_n) - f(x_{n-1})}{x_n - x_{n-1}}.$$

Substituting this into Newton's iteration gives the secant method.

Definition 2.2.17 (Secant Method)**.** Given two initial approximations x_0 and x_1, define

$$\boxed{x_{n+1} = x_n - f(x_n)\frac{x_n - x_{n-1}}{f(x_n) - f(x_{n-1})}.}$$

The denominator must be non-zero.

Remark 2.2.18 (Convergence)**.** Under suitable regularity assumptions and when the iterates approach a simple root, the secant method has order of convergence

$$\boxed{p = \frac{1+\sqrt{5}}{2} \approx 1.618.}$$

This is superlinear convergence: faster than linear convergence, but slower than the quadratic convergence of Newton's method.

Remark 2.2.19 (Advantages and Disadvantages)**.**

- Does not require an explicit derivative.
- Often converges faster than bisection.
- Usually has lower derivative-evaluation cost than Newton's method.
- Requires two initial approximations.
- Does not have the same unconditional interval guarantee as bisection.

2.2.8 Comparison of Root-Finding Methods

Method	Derivative	Convergence order	Reliability
Bisection	No	Linear	Guaranteed with sign change
Fixed-point	No	Depends on g	Depends on contraction
Newton	Yes	Quadratic locally	Initial guess matters
Secant	No	1.618 locally	Initial guesses matter

2.2.9 Problem-Solving Checklist

Remark 2.2.20 (Choosing and Applying a Method)**.**

(i) **Bisection:** check continuity and a sign change at the endpoints.

(ii) **Fixed-point iteration:** rearrange the equation as $x = g(x)$, then investigate whether g maps the interval into itself and whether $|g'(x)| < 1$.

(iii) **Newton's method:** calculate f', check that the derivative is non-zero at the iterates, and use a suitable initial guess.

(iv) **Secant method:** use two initial guesses and check that $f(x_n) - f(x_{n-1}) \neq 0$.

(v) **Error estimates:** identify the theorem's hypotheses before applying its bound.

(vi) **Convergence order:** distinguish a guaranteed convergence result from a local or asymptotic convergence result.